A AURORA AJUSTA A TRAJETÓRIA NA APROXIMAÇÃO A MARTE
Módulo De Gerenciamento De Pouso E Estabilização De Base (MGPEB)

1. DESCRIÇÃO DO CENÁRIO DE POUSO E DOS MÓDULOS DEFINIDOS

1.1 Atributos Obrigatórios por Módulo: Cada módulo possui dois conjuntos distintos de dados. A telemetria reporta o estado do módulo durante o voo (sensores, temperatura, energia). Os atributos de gestão de pouso são os campos que o sistema de fila precisa para tomar decisões de autorização e ordenação.

1.1.1 Atributos obrigatórios em todos os módulos: Os atributos obrigatórios comuns a todos os módulos são apresentados no tópico 1.2.2.

1.2 Controle e Pouso (MGPEB) - NÚCLEO DO SISTEMA: É o cérebro do projeto. Responsável por gerenciar a fila de pouso, tomar decisões de autorização e monitorar todos os outros módulos.

1.2.1 Funções: Gerenciar fila de pouso; Tomar decisões (autoriza ou não); Monitorar todos os outros módulos.

1.2.2 Atributos de Gestão de Pouso: prioridade: int — nível 1 (máxima prioridade); nivel_combustivel: float (%) — verificado na regra booleana; massa: float (kg) — necessário para cálculo de trajetória; criticidade: Enum — classificado como CRITICO; horario_chegada_orbita: datetime (UTC) — usado para ordenação na fila.

1.2.3 Telemetria de Voo: altitude (float), velocidade (float), aceleracao (float), status_retrofoguetes (bool), integridade_sensores (float %), risco_pouso (Enum baixo/médio/alto), tempo_restante_pouso (float), posicao (tuple[float, float, float]).

1.3 Energia: Sem ele, nada funciona. Alta prioridade. Responsável por gerar (solar/nuclear) e distribuir energia para toda a base.

1.3.1 Funções: Gerar energia (solar/nuclear); Distribuir energia para a base.

1.3.2 Atributos de Gestão de Pouso: prioridade: int — nível 3; nivel_combustivel: float (%); massa: float (kg); criticidade: Enum — classificado como ALTO; horario_chegada_orbita: datetime (UTC).

1.3.3 Telemetria de Pouso: nivel_energia (float %), geracao_energia (float kW), consumo_energia (float kW), eficiencia_paineis (float %), status_baterias (Enum ok/alerta/critico), temperatura_sistema (float).

1.4 Habitação: Onde os humanos ficam. Altamente crítico. Responsável pelo suporte à vida e controle ambiental interno.

1.4.1 Funções: Suporte à vida; Controle ambiental interno.

1.4.2 Atributos de Gestão de Pouso: prioridade: int — nível 4 nível 4 (alta criticidade na operação, mas com prioridade de pouso definida pela equipe); nivel_combustivel: float (%); massa: float (kg); criticidade: Enum — classificado como CRITICO; horario_chegada_orbita: datetime (UTC).

1.4.3 Telemetria de Voo: nivel_oxigenio (float %), pressao_interna (float), temperatura_interna (float), umidade (float), numero_tripulantes (int), consumo_oxigenio (float), status_suporte_vida (Enum).

1.5 Médico: Segurança da tripulação. Responsável por monitoramento de saúde e atendimento emergencial.

1.5.1 Funções: Monitoramento de saúde; Atendimento emergencial.

1.5.2 Atributos de Gestão de Pouso: prioridade: int — nível 2; nivel_combustivel: float (%); massa: float (kg); criticidade: Enum — classificado como ALTO; horario_chegada_orbita: datetime (UTC).

1.5.3 Telemetria de Voo: status_tripulacao (Enum ok/alerta/emergencia), batimentos_medios (float), nivel_estresse_tripulacao (float), estoque_medicamentos (float %), numero_pacientes (int).

1.6 Científico: Justifica a missão. Responsável pela análise de solo, atmosfera e coleta de dados científicos.

1.6.1 Funções: Análise do solo e atmosfera; Coleta e armazenamento de amostra.

1.6.2 Atributos de Gestão de Pouso: prioridade: int — nível 5; nivel_combustivel: float (%); massa: float (kg); criticidade: Enum — classificado como MEDIO; horario_chegada_orbita: datetime (UTC).

1.6.3 Telemetria de Voo: status_experimentos (Enum ativo/pausado/concluido), dados_coletados (float volume), qualidade_amostras (float), temperatura_laboratorio (float), energia_consumida (float).

1.7 Logística e Suprimentos: Sustentabilidade da base. Responsável por armazenar recursos e distribuir suprimentos.

1.7.1 Funções: Armazenar recursos; Distribuir suprimentos.

1.7.2 Atributos de Gestão de Pouso: prioridade: int — nível 6; nivel_combustivel: float (%); massa: float (kg); criticidade: Enum — classificado como MEDIO; horario_chegada_orbita: datetime (UTC).

1.7.3 Telemetria de Voo: estoque_agua (float %), estoque_alimentos (float %), estoque_equipamentos (int), taxa_consumo_diario (float), capacidade_restante (float).

1.8 Mineração: ESG e sobrevivência no longo prazo. Responsável por extrair recursos diretamente de Marte.

1.8.1 Funções: Extração de recursos locais (água, ferro, silício).

1.8.2 Atributos de Gestão de Pouso: prioridade: int — nível 7; nivel_combustivel: float (%); massa: float (kg); criticidade: Enum — classificado como BAIXO; horario_chegada_orbita: datetime (UTC).

1.8.3 Telemetria de Voo: taxa_extracao (float), tipo_recurso (Enum agua/ferro/silicio/outro), eficiencia_operacional (float), desgaste_equipamentos (float), energia_consumida (float).

1.9 Comunicação: Conexão com a Terra. Responsável por enviar e receber todos os dados da missão.

1.9.1 Funções: Enviar e receber dados com a Terra.

1.9.2 Atributos de Gestão de Pouso: prioridade: int — nível 8; nivel_combustivel: float (%); massa: float (kg); criticidade: Enum — classificado como ALTO; horario_chegada_orbita: datetime (UTC).

1.9.3 Telemetria de Voo: sinal_comunicacao (float %), atraso_comunicacao (float - delay em segundos), taxa_transmissao (float), pacotes_perdidos (int), status_antena (Enum online/offline/instavel).

1.10 VARIÁVEIS GLOBAIS DO SISTEMA - Além da telemetria e dos atributos por módulo, o sistema precisa de variáveis globais que alimentam diretamente a lógica booleana de autorização. Elas são monitoradas continuamente e atualizadas antes de cada avaliação na fila.

- condicoes_atmosfericas: Enum — valores: boa / ruim / tempestade;
- disponibilidade_area_pouso: bool — True se a área está livre;
- numero_modulos_em_pouso: int — quantos módulos estão em processo de pouso no momento.
- limite_pousos_simultaneos: int — padrão 1, configurável;
- nivel_combustivel: float (%) — do módulo atualmente em avaliação;
- prioridade_modulo: int — do módulo atualmente em avaliação.

1.10.1 Exemplo da regra booleana de autorização:

```
“POUSO_AUTORIZADO = (condicoes_atmosfericas == "boa"
AND disponibilidade_area_pouso == True
AND nivel_combustivel > 20
AND integridade_sensores > 80)”
```

1.11 Estrutura de fila de Pouso - A fila é implementada como uma classe própria (class Fila) que encapsula uma list do Python, com métodos enqueue() para inserção no final e dequeue() para remoção no início. A disciplina FIFO (First In, First Out) é a única adequada ao contexto: o primeiro módulo a chegar em órbita é o primeiro avaliado. A escolha pela implementação manual, em vez de usar collections.deque, foi pedagógica: explicitar o comportamento FIFO no código, demonstrando o conceito da estrutura de dados sem depender de uma classe pronta da biblioteca padrão. As alternativas foram descartadas: um dicionário não oferece ordem garantida; e uma pilha (LIFO) inverteria a precedência, fazendo o último a chegar ser o primeiro avaliado, comportamento incorreto. A classe Fila mantém a ordem de chegada e torna o fluxo de avaliação previsível.

1.11.1 Estrutura de um módulo completo:

```
from enum import Enum

class Criticidade(Enum):
    CRITICO = 1
    ALTO = 2
    MEDIO = 3
    BAIXO = 4

# Os 8 módulos da colônia, com listas paralelas
# (mesma posição em cada lista = mesmo módulo)
modulos = [
    "MGPEB", "Energia", "Habitacao", "Medico",
    "Cientifico", "Logistica e Suprimentos",
    "Mineracao", "Comunicacao"
]
prioridades       = [1, 3, 4, 2, 5, 6, 7, 8]
nivel_combustivel = [100, 85, 75, 90, 80, 70, 65, 95]
```

1.11.2 Criação e operação da fila:

```
# Criar a fila
fila_pouso = Fila()

# Adicionar modulos (ordem de chegada em orbita)
for nome in modulos:
    fila_pouso.enqueue(nome)

# Ver o proximo sem remover
proximo = fila_pouso.itens[0]

# Retirar para avaliacao
em_avaliacao = fila_pouso.dequeue()

# Verificar tamanho
print(f"Modulos na fila: {fila_pouso.size()}")
```

1.11.3 Exemplo de estado da fila em um dado momento (ordem de avaliação, do próximo ao último):

- Posição 1 - MGPEB | prioridade: 1 | criticidade: CRITICO | chegada: 14:30 UTC | combustível: 100%;
- Posição 2 - Médico | prioridade: 2 | criticidade: ALTO | chegada: 14:45 UTC | combustível: 90%;
- Posição 3 - Energia | prioridade: 3 | criticidade: ALTO | chegada: 15:00 UTC | combustível: 85%;
- Posição 4 - Científico | prioridade: 5 | criticidade: MEDIO | chegada: 15:20 UTC | combustível: 80%.

1.12 Listas auxiliares de estado - Além da fila, o sistema utiliza uma lista encadeada (linked list) para representar a sequência de chegada dos módulos em órbita com flexibilidade de inserção. Uma lista encadeada é composta por nós (Node) onde cada nó guarda três informações: o módulo, o horário de chegada e uma referência (ponteiro) para o próximo nó. A operação push_front() insere um novo nó no início da lista em tempo O(1), retornando a nova cabeça (head). A função print_list() percorre toda a lista do início ao fim, imprimindo cada módulo em sequência.

1.12.1 Implementação:

fila_pouso — deque - Contém os módulos que ainda estão aguardando avaliação e autorização de pouso. É a lista principal do sistema, ordenada por prioridade e horário de chegada. Operações principais: appendleft() para entrada, pop() para saída na avaliação.
modulos_pousados — list[dict] - Contém os módulos que concluíram o pouso com sucesso. Funciona como registro histórico imutável da sequência de pousos realizados. Operações principais: append() para inserir, len() para contar, iteração para relatórios.
modulos_em_alerta — list[dict] - Contém os módulos que falharam na avaliação ou foram detectados com condições críticas. Ficam em espera para reavaliação ou intervenção manual. Operações principais: append() para inserir, remove() para retirar quando resolvido.

1.12.1 Implementação:

```
class Node:
    def __init__(self, modulo, horario_chegada_orbita):
        self.modulo = modulo
        self.horario_chegada_orbita = horario_chegada_orbita
        self.next = None

def push_front(head, modulo, horario_chegada_orbita):
    novo_no = Node(modulo, horario_chegada_orbita)
    novo_no.next = head
    return novo_no

def print_list(head):
    atual = head
    while atual is not None:
        print(atual.modulo, end=" -> ")
        atual = atual.next
    print("END")
```

1.13 Pilha de Histórico de Decisões (Stack) - Para registrar o histórico de decisões e simular operações em estrutura LIFO, o MGPEB utiliza uma classe Pilha que segue a disciplina Last In, First Out: o último elemento inserido é o primeiro a ser retirado. Essa característica é ideal porque a decisão mais recente é sempre a mais relevante para consulta imediata. A classe oferece três operações fundamentais: push() empilha um item no topo, pop() remove e retorna o item do topo, e peek() consulta o topo sem remover.

1.13.1 Implementação:

```
class Pilha:
    def __init__(self):
```

```
        self.itens = []

    def push(self, item):
        self.itens.append(item)

    def pop(self):
        if self.itens:
            return self.itens.pop()
        return None

    def peek(self):
        if self.itens:
            return self.itens[-1]
        return None
```

1.14 Fluxo do sistema - Ciclo de vida do pouso: O ciclo principal de demonstração do MGPEB é executado no bloco if __name__ == "__main__" do código e percorre sete etapas sequenciais que exercitam todas as estruturas e algoritmos do sistema:
Etapa 1 - Cadastro dos módulos: o sistema lista os 8 módulos cadastrados, exibindo prioridade e nível de combustível de cada um.
Etapa 2 - Fila de pouso (FIFO): todos os módulos são inseridos na fila via enqueue(). O sistema mostra o tamanho da fila, identifica o próximo a ser avaliado e simula a retirada de 3 módulos via dequeue().
Etapa 3 - Lista encadeada: os módulos são inseridos um a um no início da lista via push_front(), e a lista é percorrida com print_list() para mostrar a sequência completa.
Etapa 4 - Pilha (LIFO): todos os módulos são empilhados via push(). O sistema consulta o topo com peek() e depois desempilha todos com pop(), simulando o pouso na ordem inversa de chegada.
Etapa 5 - Algoritmos de busca: o sistema localiza o módulo com menor combustível, o de maior prioridade e busca por um tipo específico de carga (suprimentos).
Etapa 6 - Ordenação: a lista de módulos é reordenada por prioridade usando Selection Sort, gerando a fila final de avaliação.
Etapa 7 - Autorização de pouso: o sistema percorre a fila ordenada, aplicando a função autorizar_pouso() em cada módulo com base nas variáveis globais simuladas (atmosfera, área disponível e integridade dos sensores). Cada decisão é exibida no terminal.

1.14.1 Implementação do ciclo principal:

```
def autorizar_pouso(prioridade, combustivel,
                    condicoes_atmosfericas,
                    area_disponivel,
                    integridade_sensores):
    # Vetos absolutos (Bloco 3)
    if condicoes_atmosfericas != "boa":
        return "POUSO BLOQUEADO (condicoes atmosfericas ruins)"
    elif not area_disponivel:
        return "POUSO BLOQUEADO (area indisponivel)"
    elif integridade_sensores <= 80:
        return "POUSO BLOQUEADO (sensores comprometidos)"
    # Override de emergencia (Bloco 2)
    elif combustivel < 20:
        return "POUSO IMEDIATO (combustivel critico)"
    # Condicoes normais (Bloco 1)
    elif prioridade <= 2:
        return "POUSO IMEDIATO (alta prioridade)"
    else:
        return "AGUARDAR NA FILA"
```

Resumo do ciclo: os módulos cadastrados -> entram na fila → são processados pelas estruturas (lista encadeada e pilha) → são localizados por buscas → são reordenados por prioridade → e finalmente recebem a decisão de autorização de pouso.

2. DIAGRAMA(S) DE PORTAS LÓGICAS COM AS PRINCIPAIS REGRAS DE DECISÃO

O simulador utilizado para a construção dos circuitos de portas lógicas foi o CircuitVerse (Digital Circuit Simulator online). Os circuitos desenvolvidos encontram-se disponíveis no repositório do GitHub, apresentados nos anexo 1 e anexo 2 deste relatório.

2.1 - Mapeamento das variáveis

Var	Significado	Var	Significado
A	Terreno OK	**F**	Tempestade de poeira forte
B	Modo de pouso OK	**G**	Combustível acabando
C	Grupo de sensores 1	**H**	Comunicação com a Terra falhou
D	Grupo de sensores 2	**I**	Motor superaquecido
E	Piloto automático		

Símbolos: $\wedge$ AND, $\vee$ OR, $\oplus$ XOR, $\neg$ NOT.

2.2 A Fórmula Completa:

$$\textbf{SAÍDA} = [((A \wedge B) \wedge \neg(C \oplus D) \wedge (E \oplus F)) \vee G] \wedge \neg(H \vee I)$$

2.2.1 Bloco 1 — Condições normais de pouso - $(A \wedge B) \wedge \neg(C \oplus D) \wedge (E \oplus F)$

- **$(A \wedge B)$ — porta AND -** Terreno OK **E** Modo pouso OK - Ambos precisam ser 1 ao mesmo tempo. É a condição mais básica do sistema — sem terreno seguro e sem modo de pouso ativado, nenhum pouso é cogitado.

- **$\neg(C \oplus D)$ — porta XOR + NOT (inversor) -** NÃO (Sensor 1 diferente de Sensor 2) - O XOR retorna 1 quando as entradas são diferentes. O NOT inverte: o resultado só é 1 quando os dois sensores **concordam** (ambos 0 ou ambos 1). Sensores discordantes indicam leitura inconsistente — o sistema bloqueia o pouso por segurança.

- **$(E \oplus F)$ — porta XOR -** Piloto automático OU EXCLUSIVO Tempestade de poeira forte. O XOR exige que exatamente **uma** das duas condições seja verdadeira:

  - Piloto ativo (1) **sem** tempestade (0) - pouso automatizado normal;
  - Piloto inativo (0) **com** tempestade (1) - pouso manual forçado pelo clima;
  - Piloto ativo (1) **com** tempestade (1)- risco alto sem intervenção humana;
  - Piloto inativo (0) **sem** tempestade (0) - nenhuma condição justifica o pouso.

Os três resultados são combinados com AND: **todos precisam ser verdadeiros** para o Bloco 1 valer 1.

2.2.2 Bloco 2 — Override de emergência - [Bloco 1] $\vee$ G

- **$\vee$ G — porta OR -** Bloco 1 OU Combustível acabando - Se o combustível estiver acabando (G = 1), o sistema autoriza o pouso mesmo que todas as condições normais do Bloco 1 tenham falhado. Um módulo sem combustível não pode aguardar em órbita, a perda total da nave é um risco maior do que qualquer condição adversa de pouso.

2.2.3 Bloco 3 — Vetos absolutos - ∧ ¬(H ∨ I)

- **(H ∨ I) — porta OR -** Comunicação falhou OU Motor superaquecido.
- **¬(H ∨ I) — NOT aplicado ao OR (comportamento NOR) -** NÃO (comunicação falhou OU motor superaquecido). Este bloco é um veto incondicional. Mesmo que os Blocos 1 e 2 autorizem o pouso, se H ou I for 1, a saída é forçada para 0. Esses dois problemas representam falhas catastróficas que nenhum override supera, por isso estão fora dos colchetes do Bloco 2 por design deliberado.

2.3 Verificação com os Dois Circuitos

Circuito 1 — A=1, B=1, C=0, D=0, E=0, F=1, G=0, H=0, I=0 → SAÍDA = 1 (POUSA). Bloco 1 = (1∧1) ∧ ¬(0⊕0) ∧ (0⊕1) = 1∧1∧1 = 1; ∨ G = 1; ¬(H∨I) = 1; SAÍDA = 1∧1 = 1.

Circuito 2 — A=0, B=0, C=0, D=0, E=0, F=0, G=0, H=0, I=0 → SAÍDA = 0 (NÃO POUSA). Bloco 1 = 0∧1∧0 = 0; ∨ G = 0; ¬(H∨I) = 1; SAÍDA = 0∧1 = 0.

3. IMPLEMENTAR, EM PYTHON, UM PROTÓTIPO DO MGPEB

O protótipo do MGPEB foi implementado em Python e está disponível como entregável separado deste relatório, com o código-fonte completo (.py) hospedado no repositório público do GitHub. A documentação detalhada da implementação, incluindo estruturas de dados utilizadas, algoritmos de busca e ordenação aplicados, e a lógica de simulação da autorização de pouso — encontra-se no arquivo README.md do próprio repositório, garantindo que a descrição técnica acompanhe diretamente a versão atual do código.

Repositório:
https://github.com/MaylVrosa/FASE-2-Modulo-De-Gerenciamento-De-Pouso-Estabilizacao-De-Base-MGPEB-

4. MODELAGEM DAS FUNÇÕES MATEMÁTICAS ESCOLHIDAS, COM EXPLICAÇÕES E ANÁLISES:

Durante a fase final de descida dos módulos da Missão Aurora Siger, o sistema de propulsão realiza uma frenagem controlada com o objetivo de reduzir a velocidade de descida e garantir um pouso seguro. Esse fenômeno pode ser modelado por uma função quadrática que descreve a altitude do módulo em função do tempo:

$$h(t) = h0 + v0 \cdot t - (1/2) \cdot a \cdot t^2$$

Nessa equação, h(t) representa a altitude do módulo no instante t, medida em metros. O termo h0 corresponde à altitude inicial no momento em que os retrofoguetes são ativados. A variável v0 representa a velocidade vertical inicial do módulo, assumindo valor negativo, uma vez que o movimento ocorre no sentido descendente. Já o parâmetro a representa a desaceleração provocada pelos retrofoguetes, enquanto t corresponde ao tempo decorrido desde a ativação, em segundos.

Conforme representado no Gráfico 1 (Anexo 3), com acesso por meio do link incorporado no título, a função apresenta uma parábola com concavidade voltada para baixo. Esse modelo é válido para $t \geq 0$ até o instante em que $h(t) = 0$, que representa o momento do contato com o solo.

Como exemplo prático, considerando h0 = 800 metros, v0 = −40 m/s e a = 6 m/s², a altitude no instante t = 5 s é de 525 metros. O instante de pouso pode ser determinado ao resolver a equação h(t) = 0, caracterizando um problema de segundo grau.

Do ponto de vista gráfico, a função descreve uma parábola com concavidade voltada para baixo, indicando que a altitude diminui ao longo do tempo, mas com redução progressiva da

velocidade de descida, devido à desaceleração causada pelos retrofoguetes. À medida que o tempo aumenta, a altitude se aproxima de zero, sinalizando a aproximação do pouso.

Essa modelagem permite que o MGPEB (Módulo de Gerenciamento de Pouso e Estabilização da Base) calcule antecipadamente o momento ideal para intensificar a atuação dos retrofoguetes. Dessa forma, evita-se tanto o acionamento prematuro, que levaria ao desperdício de combustível, quanto o acionamento tardio, que poderia resultar em uma velocidade de impacto perigosa. Assim, a função matemática está diretamente integrada à lógica de decisão do sistema.

O combustível dos módulos constitui um recurso crítico e limitado. Durante a fase de descida, quanto maior for a velocidade que precisa ser reduzida, maior será o consumo dos propulsores. Essa relação pode ser representada, de forma simplificada, por uma função linear:

$$C(v) = k \cdot v$$

Conforme representado no Gráfico 2 (Anexo 4), com acesso por meio do link incorporado no título, a zona vermelha indica consumo acima de 100 kg, situação em que o MGPEB pode bloquear o pouso.

Nessa equação, C(v) representa o consumo de combustível durante a manobra de frenagem, v corresponde à velocidade de entrada na fase de descida (em m/s) e k é o coeficiente de consumo específico do propulsor, que depende das características do sistema. Esse modelo linear é uma aproximação adequada para fins de tomada de decisão, embora, na prática, o consumo possa depender de outros fatores, como a massa do módulo e a eficiência dos motores.

Por exemplo, considerando k = 1,5 kg por m/s, uma entrada com velocidade de 40 m/s resulta em um consumo de 60 kg de combustível, enquanto uma entrada a 80 m/s resulta em 120 kg. Graficamente, a função é representada por uma reta que parte da origem, indicando uma relação de proporcionalidade direta: à medida que a velocidade aumenta, o consumo cresce de forma linear.

Essa função permite analisar o comportamento do sistema: quanto maior a velocidade inicial, maior será o consumo necessário para garantir uma frenagem segura. Além disso, o coeficiente k influencia diretamente a inclinação da reta — valores maiores indicam menor eficiência do propulsor.

O MGPEB utiliza a função C(v) para verificar, antes de autorizar o pouso, se o combustível disponível no módulo é suficiente para realizar a desaceleração necessária. O consumo estimado é comparado ao nível de combustível atual: caso C(v) seja maior que o combustível restante, o sistema sinaliza uma condição de risco e bloqueia a tentativa de pouso, conectando diretamente a modelagem matemática à regra de decisão (nível de combustível > 20%).

Dessa forma, a utilização de funções matemáticas permite transformar variáveis físicas em critérios objetivos de decisão, conectando diretamente a modelagem matemática às estratégias de engenharia adotadas no sistema de pouso.

5. SEÇÃO DE CONTEXTUALIZAÇÃO HISTÓRICA/ARQUITETURAL DO SISTEMA

É curioso perceber como a mesma necessidade humana, contar, calcular, prever, atravessa séculos sem mudar de essência. Foi ela que levou Blaise Pascal a construir uma máquina de somar no século XVII, e é essa mesma lógica que hoje nos desafia a programar sistemas capazes de sobreviver numa missão a Marte.

A história dos computadores, nesse sentido, não é apenas uma sequência de invenções. É o reflexo de uma ambição constante: fazer mais com menos, com mais precisão e com menos margem para erro.

A trajetória dos computadores é uma sequência contínua de miniaturização: começou com válvulas eletrônicas (1ª geração, 1945–1959, o ENIAC pesava 30 toneladas e tinha 17.840 válvulas que queimavam constantemente), evoluiu para transistores (2ª geração, 1959–1964, peças que operam como chaves eletrônicas, o mesmo princípio das portas AND, OR e NOT usadas no MGPEB), depois para circuitos integrados com as técnicas LSI, VLSI e ULSI (3ª, 4ª e 5ª gerações),

que permitiram passar de dezenas a milhões de transistores em um único chip. A Lei de Moore (Gordon Moore, Intel) descreveu com precisão esse ritmo durante décadas: o número de transistores por processador dobra a cada 18 meses.

O que mudou, ao longo das décadas, foi a escala. Os computadores tornaram-se menores e mais eficientes, até chegarem aos sistemas embarcados que hoje operam em satélites e naves. As missões do Apollo Program ilustram bem esse ponto. O Apollo Guidance Computer operava com apenas alguns kilobytes de memória, muito menos do que qualquer dispositivo trivial de hoje. Uma curiosidade técnica é que seu software era a Core Rope Memory: fios de cobre "tecidos" à mão por dentro de anéis magnéticos para evitar que o código fosse apagado por vibrações ou radiação.

Programas mais recentes, como o Artemis Program, contam com sistemas mais avançados, mas os princípios permanecem os mesmos: o espaço pune qualquer descuido. Um desafio moderno é que não podemos usar os chips mais rápidos da Terra em Marte. Devido à radiação intensa, que pode causar o bit-flip (inversão de dados no chip), a NASA utiliza processadores Radiation-Hardened (Rad-Hard). Eles são fisicamente mais robustos e "lentos" que um celular atual, mas garantem que o sistema não trave no meio de uma manobra crítica.

Quando se fala numa missão a Marte, como no cenário da base Aurora, esses desafios tornam-se ainda mais exigentes. A distância, por si só, elimina qualquer possibilidade de controle em tempo real: um sinal pode levar entre 3 e 22 minutos a percorrer o trajeto entre os dois planetas. Na prática, isso obriga o sistema a funcionar de forma autônoma, resiliente e confiável por conta própria.

As limitações de hardware passam a ditar cada escolha. A memória é escassa, o que exige estruturas de dados compactas e previsíveis. O poder de processamento é limitado, tornando inviáveis algoritmos de alta complexidade. O consumo de energia precisa ser rigorosamente controlado, cada watt desperdiçado impacta diretamente sistemas críticos. E há ainda a radiação, capaz de corromper dados e danificar circuitos, tornando a redundância e a tolerância a falhas requisitos básicos, e não apenas boas práticas.

Tudo isso se traduz em decisões concretas de programação. Os algoritmos precisam de ser determinísticos; o sistema não pode apresentar comportamentos inesperados. As estruturas de dados devem ser previsíveis em tamanho e tempo de acesso. Estratégias como redundância de módulos, watchdogs, fail-safes e computação distribuída deixam de ser opcionais, tornam-se essenciais.

É nesse contexto que o MGPEB se insere. Mais do que uma solução técnica, ele representa uma continuidade histórica. A mesma lógica que levou engenheiros a extrair o máximo de máquinas com recursos mínimos, de Babbage ao programa Apollo, continua a orientar as decisões atuais.

A exploração espacial não tolera abstrações mal pensadas nem otimizações adiadas. Ela exige o melhor da engenharia de software, aplicado com rigor, intenção e um nível de responsabilidade proporcional ao ambiente em que opera. E é exatamente isso que o MGPEB busca ser: simples o suficiente para ser confiável, e confiável o suficiente para pousar módulos em Marte.

6. SEÇÃO DE REFLEXÃO SOBRE ESG E GOVERNANÇA NA BASE AURORA SIGER.

Construir uma base permanente em Marte exige muito mais do que resolver desafios de engenharia. Obriga a pensar em como vamos viver, e sobreviver, num ambiente extremo. Por isso, as decisões não podem ser apenas técnicas: precisam de ser orientadas por critérios de governança ambiental, social e corporativa (ESG), não como uma obrigação formal, mas como uma condição real de sustentabilidade.

A importância vinda do material é o risco do greenwashing: adotar o discurso de sustentabilidade sem a prática real. Em Marte, um sistema que declare eficiência energética sem garanti-la pode custar vidas. Portanto, os critérios ESG da base Aurora precisam ser auditáveis, registrados e verificados, não apenas declarados.

No eixo ambiental (E), a escolha do local de pouso é o primeiro grande teste. Mesmo num planeta onde não há vida conhecida, isso não significa ausência de responsabilidade. É essencial

evitar contaminação com microorganismos terrestres, preservando áreas de interesse científico que poderiam ser irrecuperáveis após o contato. A proximidade a reservas de gelo, a proteção contra radiação e a eficiência energética devem pesar tanto quanto a logística imediata, e, sempre que possível, o ideal é pousar diretamente no local definitivo da base, evitando deslocamentos adicionais e os impactos associados.

A gestão de recursos segue essa mesma lógica: energia majoritariamente solar, com sistemas de armazenamento inteligentes; reaproveitamento quase total de resíduos; e produção de alimentos em sistemas fechados dentro da própria base. A milhões de quilômetros da Terra, não há margem para desperdício, a circularidade deixa de ser uma escolha e passa a ser uma necessidade.

Já o eixo social (S) está diretamente ligado à forma como as pessoas convivem e tomam decisões. Em Marte, a sobrevivência depende de cooperação real, não apenas de boas intenções. Isso implica regras claras sobre a divisão de recursos e a distribuição de responsabilidades. Uma abordagem possível seria destinar cerca de 80% dos recursos à operação e sobrevivência da base, e 20% a um fundo comum voltado para investigação, inovação e melhoria das condições de vida. A utilização desse fundo deveria ser decidida de forma coletiva, com base em votações e acordos transparentes, sempre acompanhados de registros digitais que garantam prestação de contas. Cada membro contribui com base na sua especialização, mas dentro de limites que evitem sobrecarga e desigualdades internas.

Por fim, o eixo da governança (G) define como tudo isso se organiza na prática. Mais do que escolher entre modelos centralizados ou distribuídos, o essencial é garantir transparência, responsabilidade e equilíbrio de poder. Estruturas baseadas em especialização, como sistemas do tipo WES (Work/Expertise Systems), podem ajudar a distribuir funções de forma eficiente, mantendo autonomia técnica sem perder o controle coletivo. Diferentes modelos organizacionais podem coexistir, desde que compartilhem princípios claros e bem definidos desde o início. Esses acordos devem ser estabelecidos antes da chegada a Marte e registrados em sistemas digitais que assegurem rastreabilidade e confiança, evitando a concentração de poder em poucos.

A base Aurora pode ser mais do que um marco tecnológico. Se os critérios ESG forem aplicados de forma consistente, pode tornar-se um verdadeiro laboratório de organização humana, um espaço onde testamos não só como sobreviver fora da Terra, mas também como viver de forma mais equilibrada, justa e sustentável.